Structured output (a checked answer, not a hope) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea: a form with locked fields
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea: a form with locked fields

Two kinds of field do most of the work:

- `str | None` — the field **must** be there, but `None` is an allowed answer. A plain `str` would force the model to invent a name when the message has none (the Structured output topic above).
- `Literal["low", "normal", "urgent"]` (from `typing`) — only those exact values pass. `"very"` or `"Urgent"` raises `ValidationError`.

Then the API call: `client.beta.chat.completions.parse(model=..., messages=..., response_format=ExtractedInfo)`. It sends your class as a strict schema, and runs `model_validate` on the reply for you.

Hint 2 — The plan, and almost the whole thing

```
function extract_info(text) -> ExtractedInfo | None:
    try: completion = .parse(system prompt + text, ExtractedInfo)
    except ValidationError: return None
    message = the first choice's message
    if message.refusal: print it, return None
    info = message.parsed
    if info.name is None and info.intent is None: return None
    return info
```

Tell the model in the system prompt to use `null` instead of guessing — the `str | None` field makes that possible, the prompt makes it happen.

Returning `None` for "nothing found" gives the caller exactly two cases: an object it can trust, or `None`. Run it on a clear message, a message with no name, and `"the sky is blue"`, then compare against the Solution.

Last, let the user type the text: loop on `input("You: ")`, stop on `/quit`, and when a line `.startswith("/extract ")`, pass `line.removeprefix("/extract ")` to `extract_info()`.

Full solution: Show me the solution